

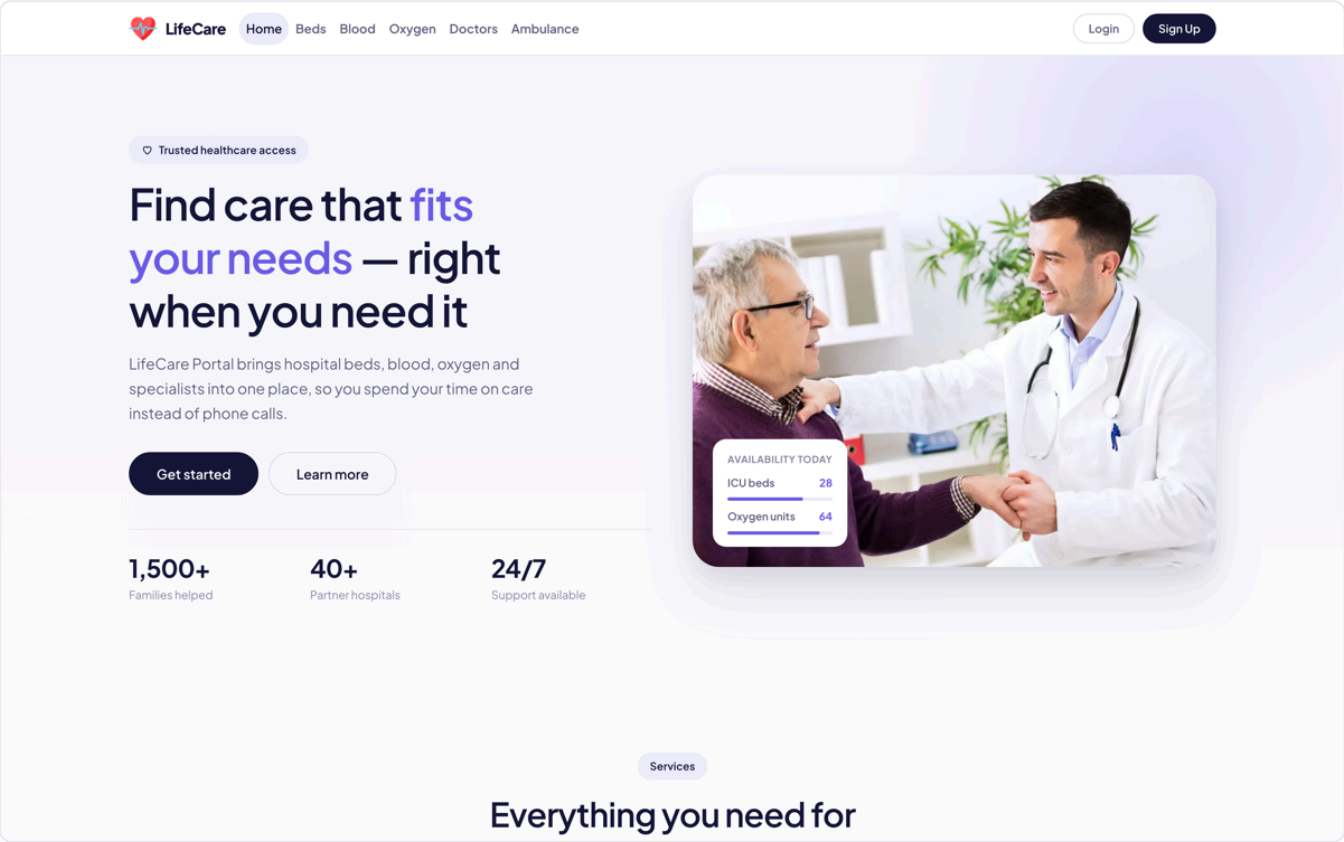
LifeCare Portal

How the system works — roles, request lifecycle, architecture and deployment.

Generated 3 September 2026 · React 19 + Vite on CloudFront · FastAPI on AWS Lambda · PostgreSQL (Neon)

1. What it does

LifeCare Portal puts hospital bed, blood and oxygen availability in one place, so someone looking for urgent care can see what is free and reserve it instead of phoning hospitals one by one. Hospitals publish their own availability; patients search and request; hospitals approve or reject. An administrator onboards the hospitals.



The public home page — what a visitor sees before signing in

2. The three roles

Every account is exactly one of three roles, decided at login. The role is carried in the JWT and enforced on the server for every protected route, so hiding a link in the UI is never the only thing standing between a user and someone else's data.

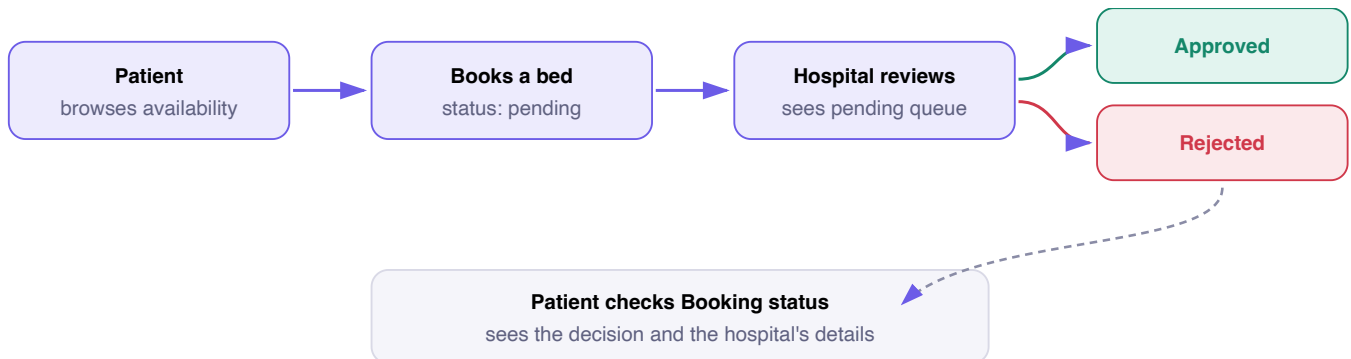
ROLE	SIGNS IN AS	CAN DO
Admin	a site administrator	Register hospitals, review and update the hospital list, browse registered users

Hospital	one hospital account	Publish bed, blood, oxygen and doctor availability; approve or reject incoming requests
Patient	a member of the public	Browse availability across hospitals, book a bed, track the request, find ambulance contacts

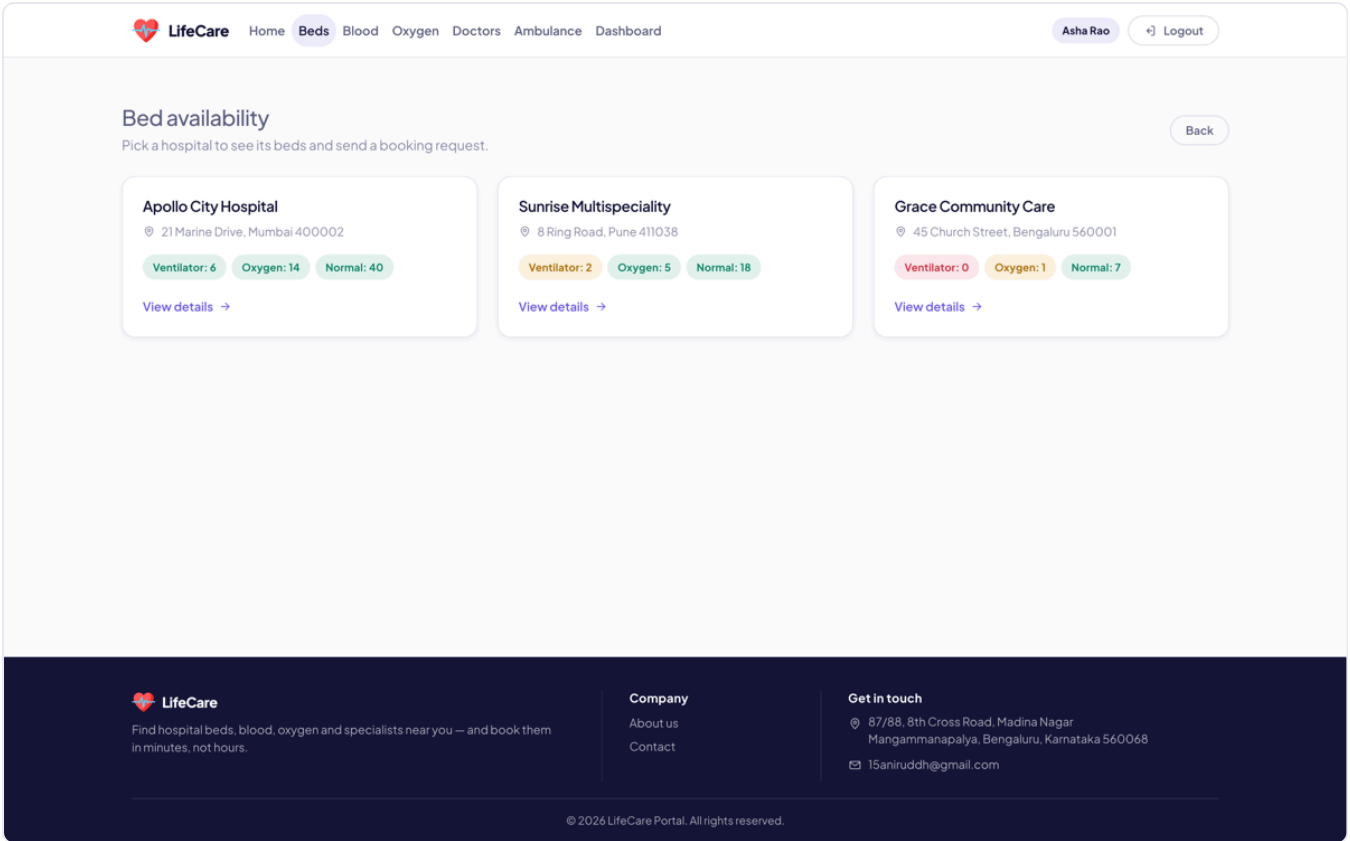
Scoping. A hospital account can only read and write its own records. The server compares the hospital id in the token against the id in the request, so one hospital cannot see or edit another's beds even by guessing a URL.

3. The core flow: booking a bed

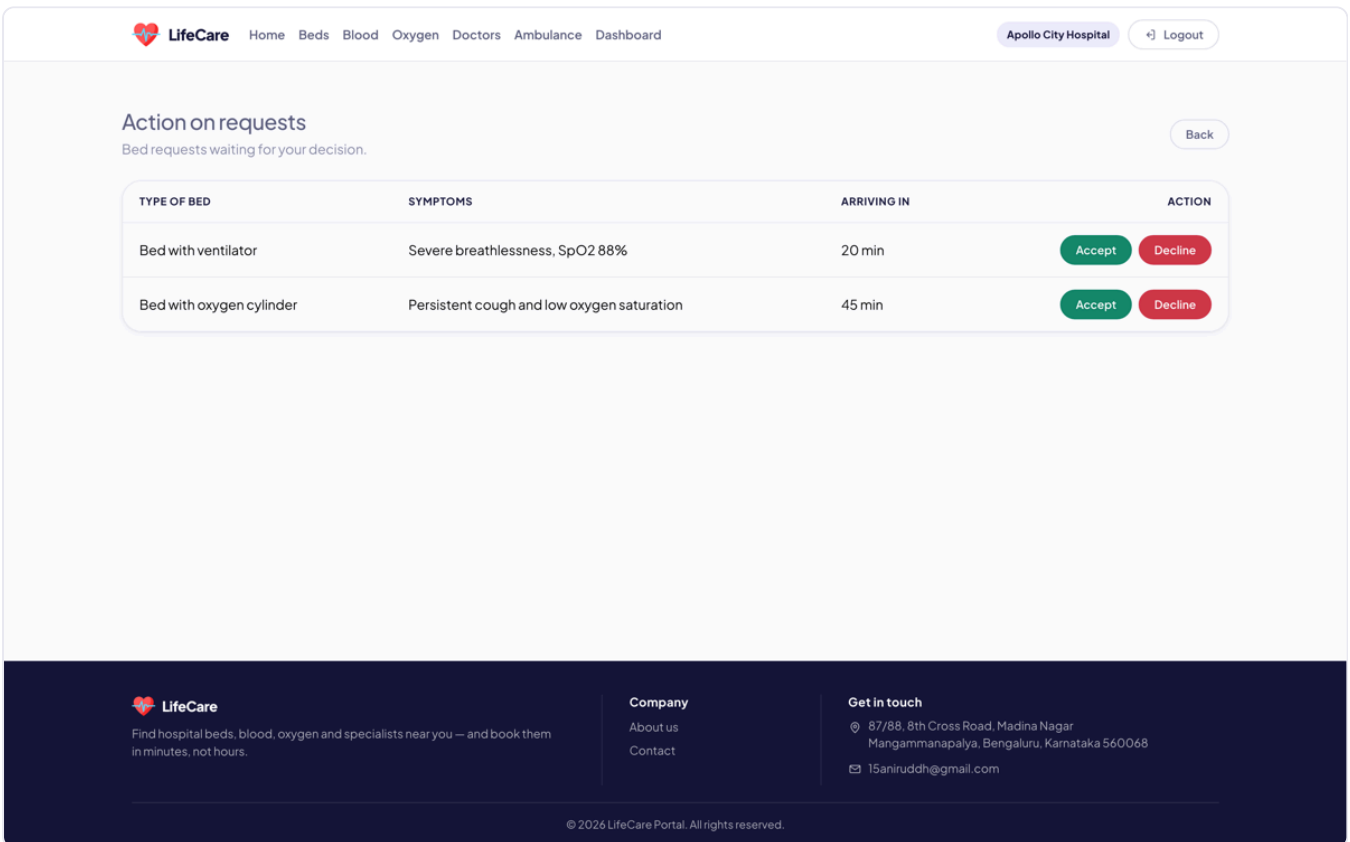
This is the path that matters most, end to end.



1. **Browse.** The patient opens Bed availability and sees every hospital's current counts, pulled live from the API.
2. **Request.** They pick a hospital and bed type, describe symptoms and give an expected arrival time. The request is written with status `pending`.
3. **Review.** That hospital — and only that hospital — sees the request in its pending queue.
4. **Decide.** Approving or rejecting writes the new status. Approval also decrements the hospital's available beds so the same bed is not promised twice.
5. **Follow up.** The patient sees the outcome under Booking status.

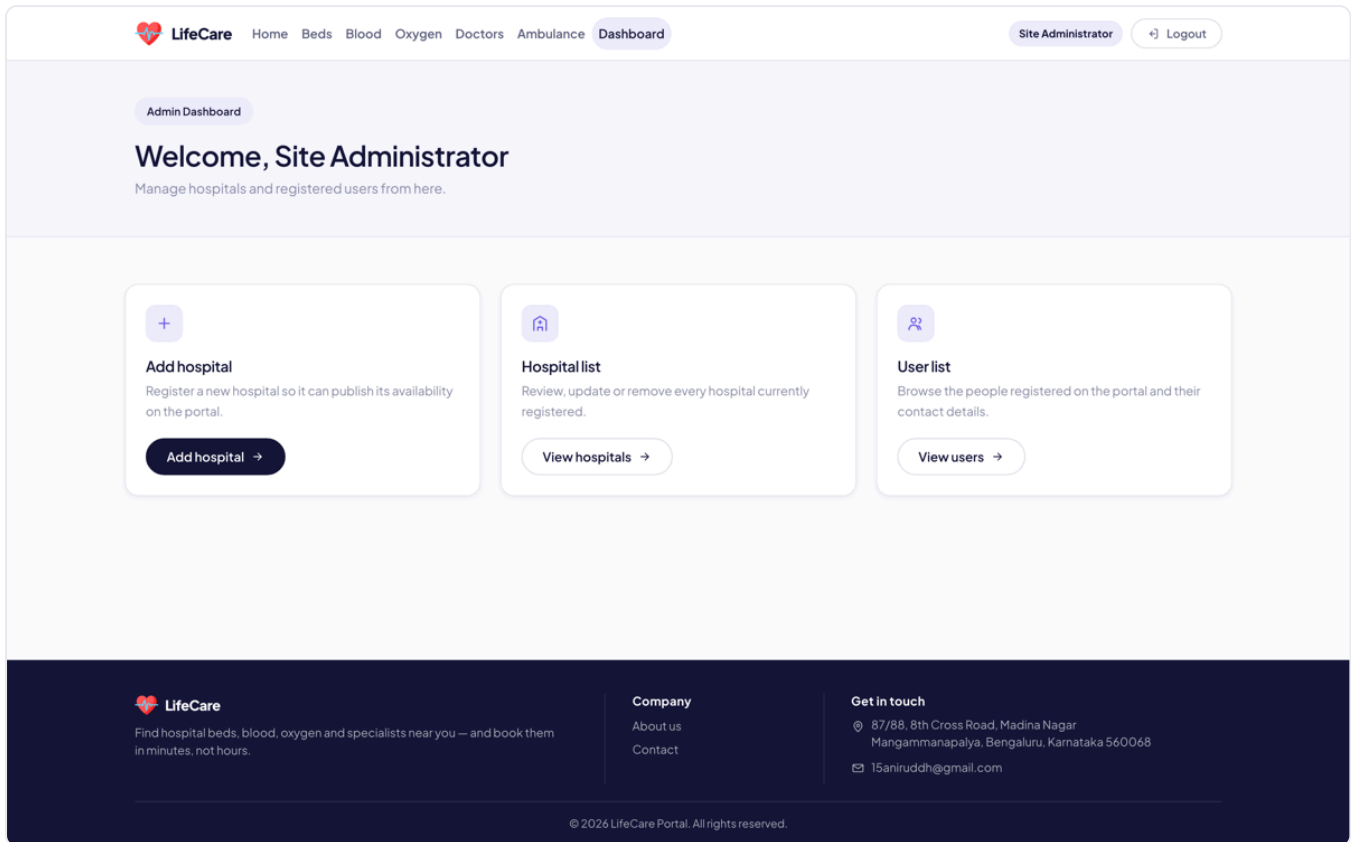


What a patient sees before booking — live counts across hospitals

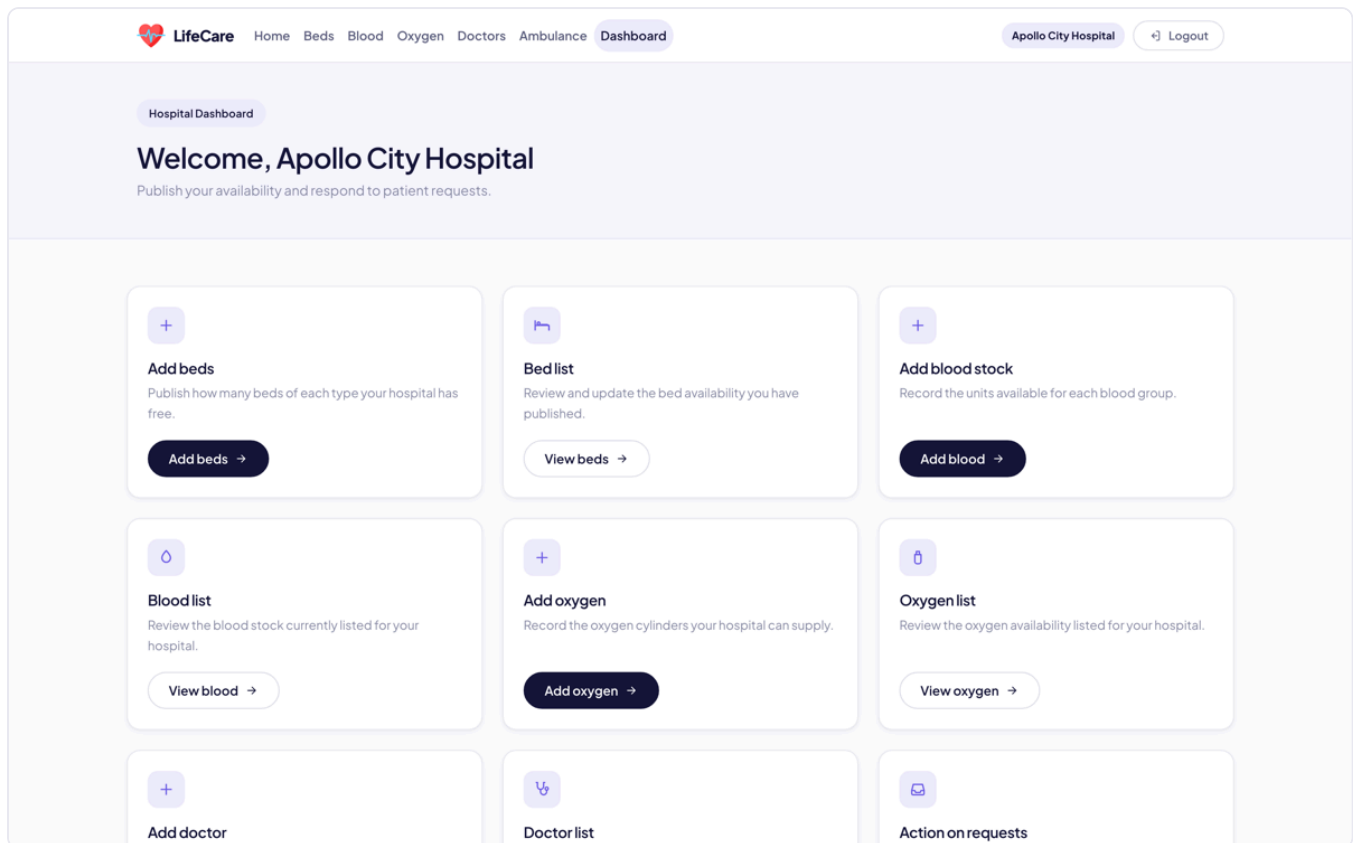


The hospital's pending queue, scoped to its own requests

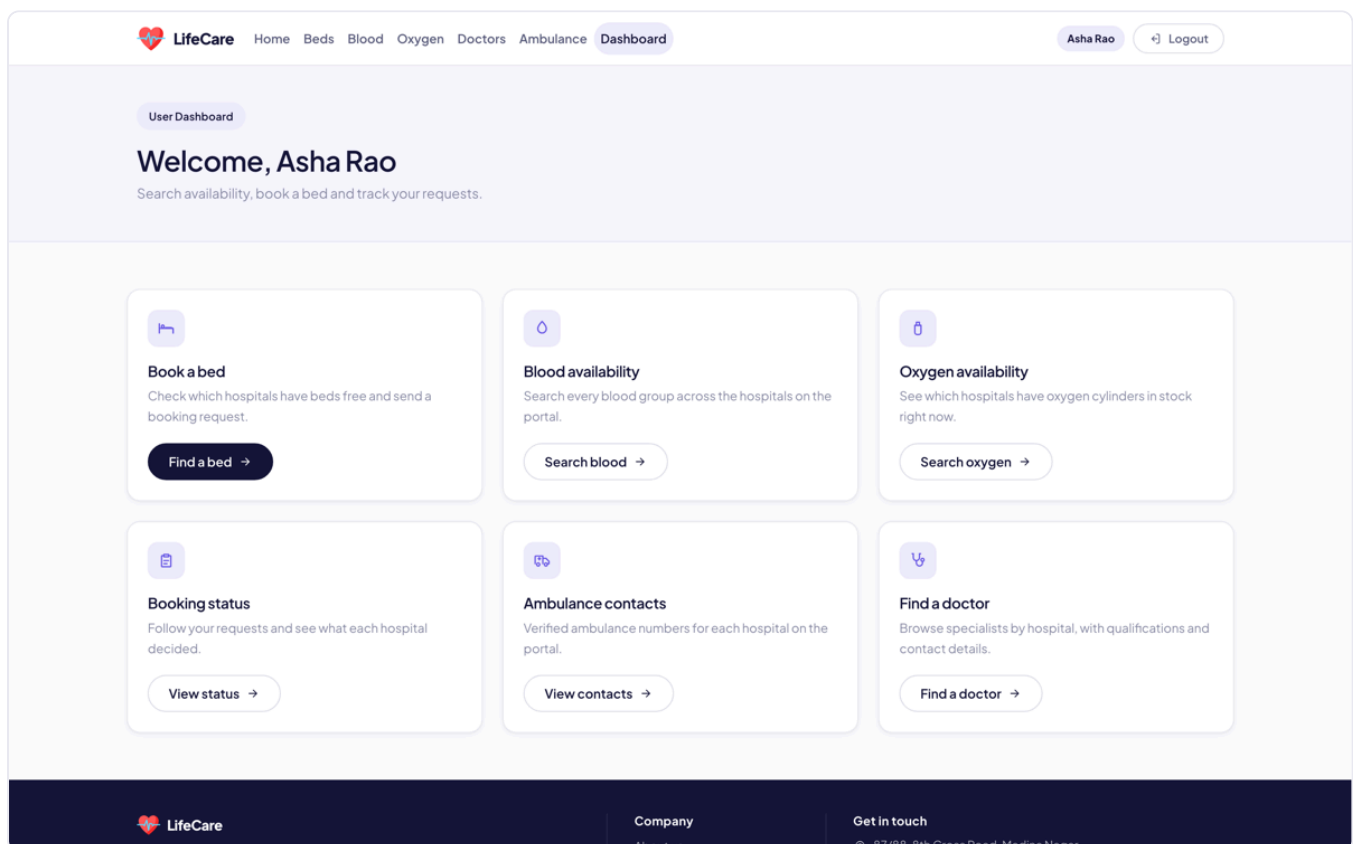
4. What each role sees



Admin — onboard hospitals and review who is registered

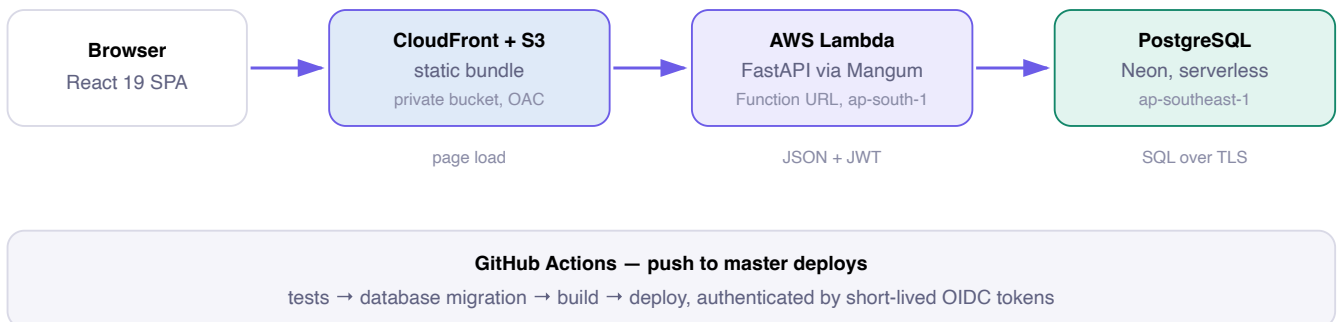


Hospital — publish availability and handle requests



Patient — search, book and track

5. How it is built



Frontend

A React 19 single-page app built by Vite. It is served as static files from a private S3 bucket through CloudFront, which also rewrites unknown paths back to `index.html` so client-side routes survive a refresh. The API address is compiled into the bundle at build time.

Backend

A FastAPI application running on a single AWS Lambda function. Mangum translates the Lambda event into the request FastAPI expects, so the same code runs locally under Uvicorn and in the cloud unchanged. A Lambda Function URL exposes it directly — there is no API Gateway in front, which keeps both the cost and the moving parts down.

Database

PostgreSQL on Neon, which scales to zero when idle. Six tables: `hospitals`, `users`, `admins`, `doctor_info`, `requests` and Alembic's `alembic_version`. Schema changes are versioned as Alembic migrations and applied by the pipeline before new code goes live.

Authentication

Signing in returns a JSON Web Token carrying the account id and role. The browser keeps it in `sessionStorage` and attaches it to every subsequent call; the server re-checks it on each protected route. "Continue with Google" is an alternative way in — the exchange happens server-side and ends by minting the same kind of token, so everything downstream is identical.

6. How a change reaches production

Pushing to `master` is the whole deployment process. Two workflows watch different folders, so a frontend change never rebuilds the backend and a documentation change deploys nothing at all.

STAGE	WHAT HAPPENS
Test	Linting, type checking and the test suites. A failure stops everything here.
Migrate	<code>alembic upgrade head</code> runs against the live database. If it fails, the deploy is skipped and the old version keeps serving.
Build	The Lambda package, or the static bundle with the API address compiled in.
Deploy	CloudFormation updates the stack; the site syncs to S3 and the CDN cache is invalidated.
Verify	The pipeline calls the health endpoint and prints the live URL.

No stored AWS keys. GitHub proves its identity with a short-lived OpenID Connect token that AWS trades for credentials lasting an hour, restricted to this repository and this branch. Nothing long-lived is kept in the repository.

7. Running costs and safety limits

The deployment is designed to cost nothing while idle: Lambda bills per request, CloudFront's free allowance covers ordinary traffic, and the database sleeps when unused. Because the API is publicly reachable, two limits guard against abuse — an alarm on an unusual spike in requests, and an automatic throttle that stops the function outright if that alarm fires. Login attempts are separately rate-limited per address.